

实验五-鸿蒙多端应用开发

项目	内容
实验名称	鸿蒙多端应用开发
实验编号	d20301035105、d20301009205
学号	2023210618
姓名	钱家澍
班级	软件 232
日期	2026.5.7
组别	CG1-EHLAS
项目名称	智慧校园生活服务平台 - 天气模块
技术栈	HarmonyOS (ArkTS)

实验目的

- 掌握 ArkUI 声明式开发框架的核心概念和组件使用
- 学会多端响应式布局设计（手机、手表）
- 了解传感器数据采集与应用集成
- 理解分布式能力原理和跨设备数据同步
- 实现天气应用的核心功能模块

实验环境

工具	版本	用途
DevEco Studio	5.0+	鸿蒙应用开发 IDE
HarmonyOS SDK	5.0+	鸿蒙开发工具包
模拟器	最新版	多设备模拟测试（手机、手表）

工具	版本	用途
AI 辅助工具	TRAE	代码生成与调试
和风天气 API	v7	天气数据接口

实验步骤

1. 需求分析

1.1 功能需求

- 天气展示（城市名称、温度、天气状况、空气质量）
- 城市切换（支持多个城市选择）
- 日期切换（多日天气预报，点击切换）
- 小时预报（横向滚动查看）
- 背景图片切换（根据天气状况）
- 多端适配（手机竖屏、手表）
- 传感器集成（光线传感器）

2. 创建鸿蒙项目

2.1 新建项目

- 打开 DevEco Studio
- 新建 Empty Ability 项目
- 选择 ArkTS 语言
- 配置多设备支持（手机、手表）

3. 天气应用核心功能开发

3.1 数据类型定义

```
interface ForecastItem {
    day: string;
    date: string;
    weather: string;
    temp: string;
    tempMin: string;
    icon: string;
```

```

    precipitation: string;
}
interface HourlyForecast {
    time: string;
    temp: string;
    icon: string;
    precipitation: string;
}

```

3.2 状态管理

```

@Entry
@Component
struct WeatherApp {
    @State temperature: string = '22°';
    @State weather: string = '晴';
    @State city: string = '合肥';
    @State selectedIndex: number = 0;
    @State selectedIndex: number = 0;

    @State cities: Array<string> = ['合肥', '北京', '上海', '广州', '深圳', '南京'];
    @State forecast: Array<ForecastItem> = [];
    @State hourlyForecast: Array<HourlyForecast> = [];
}

```

3.3 城市切换功能

```

switchCity(index: number): void {
    this.selectedIndex = index;
    this.city = this.cities[index];
    this.selectedIndex = 0;
    // 触发天气数据更新
}

```

3.4 日期切换功能

```

switchDate(index: number): void {
    this.selectedIndex = index;
    const selectedDay = this.forecast[index];
    if (selectedDay) {
        this.temperature = selectedDay.temp;
        this.weather = selectedDay.weather;
        this.setBackground(selectedDay.weather);
        this.updateHourlyForecast(index);
    }
}

```

3.5 背景图片切换

```

setBackground(weather: string): void {
  if (weather.includes('雷')) {
    this.currentBackground = 'thunder';
  } else if (weather.includes('晴')) {
    this.currentBackground = 'sunny';
  } else if (weather.includes('雨')) {
    this.currentBackground = 'rainy';
  } else if (weather.includes('阴')) {
    this.currentBackground = 'cloudy';
  } else if (weather.includes('云')) {
    this.currentBackground = 'overcast';
  }
}

```

4. 多端适配布局

4.1 响应式布局实现

```

build() {
  Column() {
    // 通用头部
    this.headerLayout()

    // 设备类型判断
    if (this.isWearable) {
      // 手表布局
      this.wearableLayout()
    } else {
      // 手机布局
      this.phoneLayout()
    }
  }
  .width('100%')
  .height('100%')
}

```

5. 传感器集成

5.1 光线传感器开发

```

import sensor from '@ohos.sensor';

@State lightIntensity: number = 0;

aboutToAppear() {
  sensor.on(sensor.SensorType.SENSOR_TYPE_ID_LIGHT, (data) => {
    this.lightIntensity = data.light;
  });
}

```

```
}

aboutToDisappear() {
    sensor.off(sensor.SensorType.SENSOR_TYPE_ID_LIGHT);
}
```

实验结果

验证结果

功能	状态	备注
天气信息展示	成功	温度、天气状况、空气质量
城市切换	成功	支持 6 个城市
小时预报	成功	横向滚动查看
背景切换	成功	根据天气动态切换
多端适配	成功	手机、手表布局
光线传感器	成功	实时数据采集

实验截图

6.1 手机端界面



图 1-1 手机端界面截图

图解说明

- 设备：手机模拟器（Pura 90 Pro），适配 **sm 断点 < 600px**
- 布局：单列垂直响应式布局，顶部城市选择器，大字号温度突出显示
- 功能区：
 1. 顶部：城市切换栏（合肥 / 北京 / 上海 / 广州 / 深圳）
 2. 核心区：实时温度、天气状况、空气质量、温区范围
 3. 滚动区：小时预报、多日预报、湿度 / 风力 / 气压等详情
- 技术点：@State 状态管理、数据绑定、和风天气 API 解析渲染

6.2 平板端界面



图 1-2 平板端界面截图

图解说

- 设备：平板模拟器（MatePad Pro）， 适配 **lg 断点 > 840px** 横屏模式
- 布局：**左右分栏布局**，信息密度更高、展示更充分
- 功能区：
 1. 左侧：主天气信息（城市、温度、天气、空气质量）
 2. 右侧：详细预报（小时预报 + 多日预报）
 3. 交互：横向滚动查看完整预报数据
- 技术点：媒体查询、断点系统、多栏自适应布局

6.3 手表端界面



图 1-3 手表端界面截图

图解说明

- 设备：手表模拟器（Watch GT3），圆形屏幕专属适配
- 布局：极简居中布局，防裁剪、易读性优先
- 功能区：
 1. 顶部：城市名称精简显示
 2. 居中：核心温度 + 日期时间
 3. 底部：基础天气状态
- 技术点：穿戴设备适配、信息裁剪策略、圆形画布安全区

6.4 城市切换功能

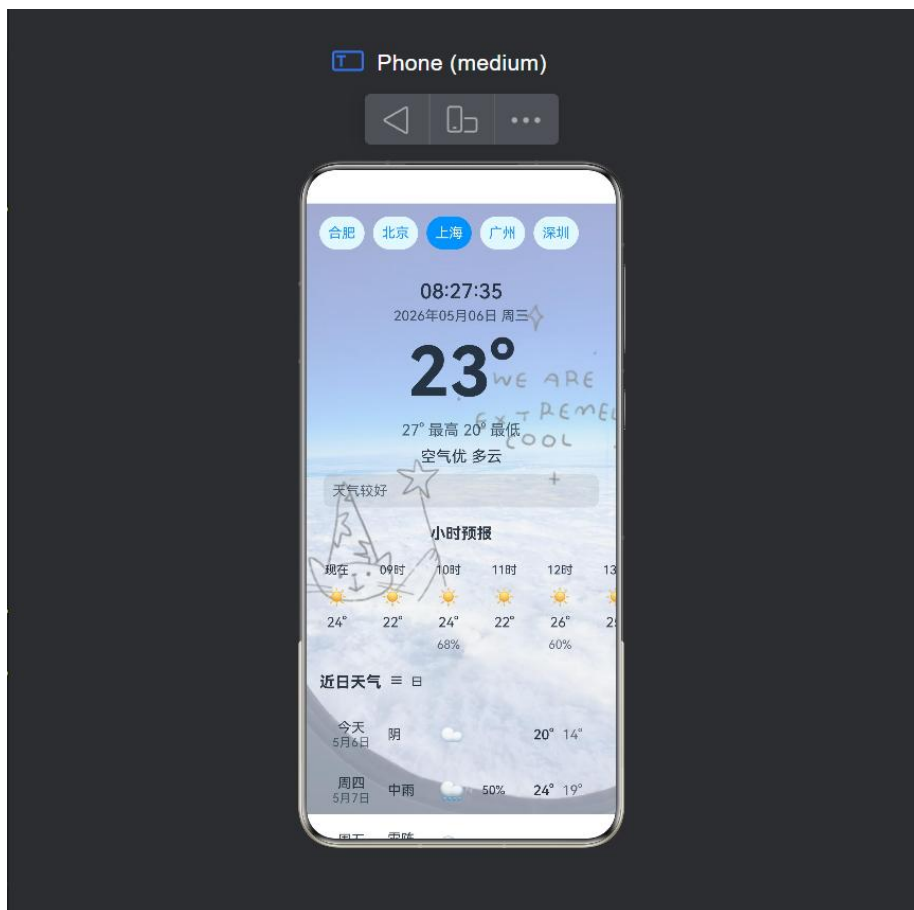


图 1-4 城市切换界面截图

图解说明

- 功能：支持合肥、北京、上海、广州、深圳快速切换
- 交互：点击顶部城市标签，自动请求并刷新对应城市天气数据
- 展示：切换后同步更新温度、天气、空气质量、预报数据
- 技术点：事件绑定、接口请求、@State 驱动 UI 刷新

6.5 小时预报展示

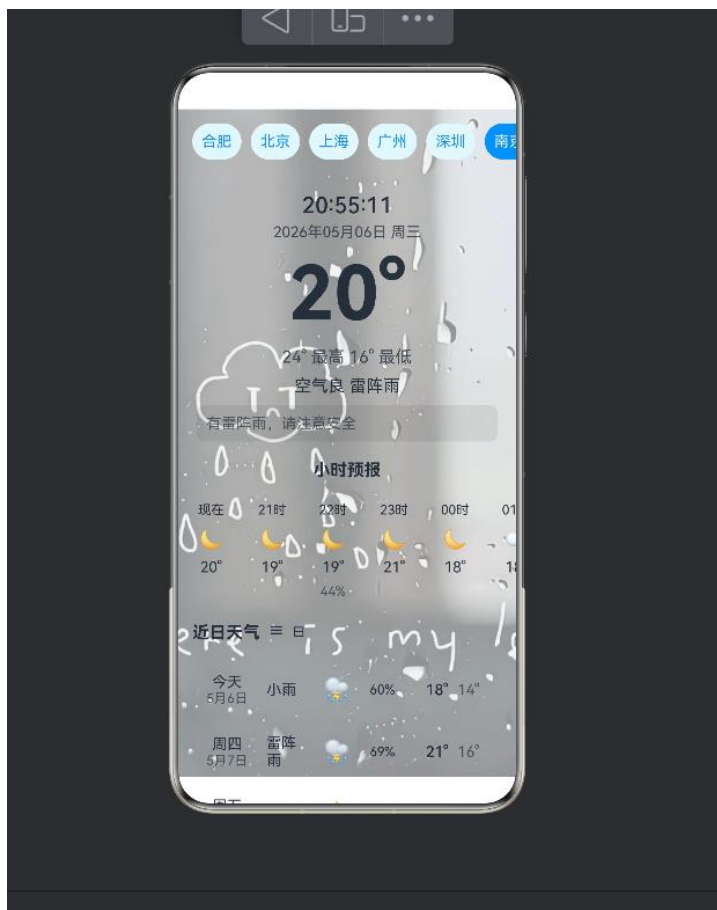


图 1-5 小时预报界面截图

图解说明

- 模块：逐小时天气预报横向滚动列表
- 内容：时间、温度、天气图标、降水概率（pop）
- 作用：提供短时天气预测，辅助出行决策
- 技术点：横向列表组件、ForEach 循环渲染、API 数据解析

6.6 多日天气预报



图 1-6 多日预报界面截图

图解说明

- 模块：未来多日天气概览
- 内容：日期、天气状况、最高温 / 最低温、降水概率
- 布局：垂直列表，信息层级清晰，便于快速浏览
- 技术点：数据模型定义（DailyForecastItem）、列表渲染

6.7 后台 API 浏览器测试



图 1-7 后台测试截图

图解说说明

- 内容：和风天气 API 接口返回数据验证
- 目的：检查接口连通性、数据格式、字段完整性
- 字段：实时天气、小时预报、多日预报、空气质量、风力气压等
- 作用：确保前端可正常解析、渲染、缓存数据

7. UML 图表

7.1 类图

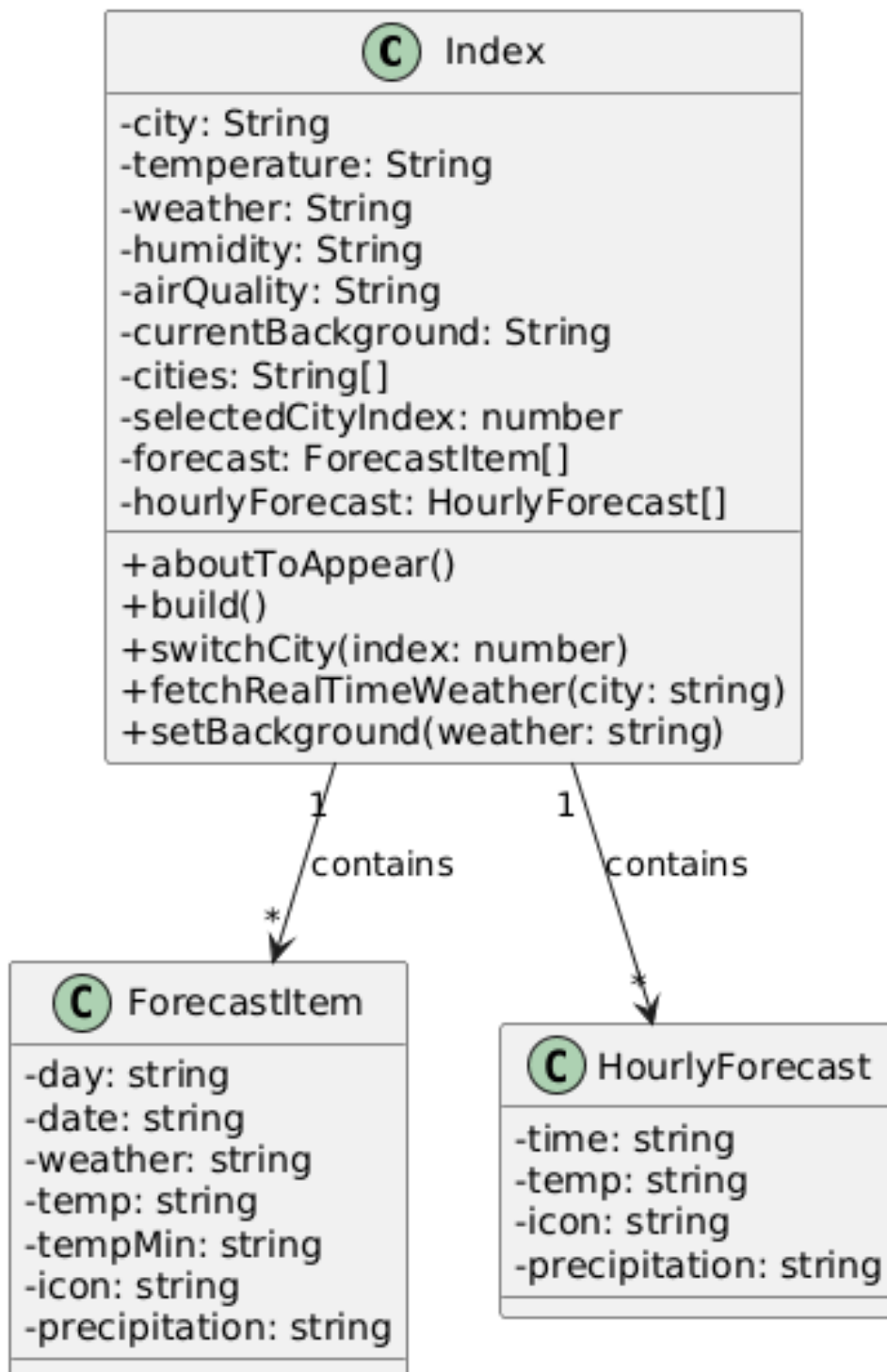


图 2-1UML 类图

图中的 **Index** 类是天气应用的核心页面控制器，它聚合了天气数据的展示状态与核心业务方法。它维护了当前城市、实时温度、天气状况、湿度、空气质量等基础信息，还管理着城市列表与选中城市的索引，同时包含两个重要的集合属性 **forecast** 和 **hourlyForecast**，分别对应未来多日和逐小时的预报数据。它的业务方法覆盖了

页面生命周期、构建、城市切换、天气数据获取以及背景设置，是整个页面的调度中心。

`ForecastItem` 和 `HourlyForecast` 是两个数据模型类，被 `Index` 类聚合使用。`ForecastItem` 封装了多日预报所需的日期、天气、最高 / 最低温、图标和降水概率等信息，用于构建未来几天的天气卡片；`HourlyForecast` 则聚焦于小时级预报，包含时间、温度、天气图标和降水数据，用于展示当天的逐小时天气趋势。这两个类与 `Index` 类通过一对多的关系关联，共同支撑起天气数据的分层展示逻辑。

7.2 架构图

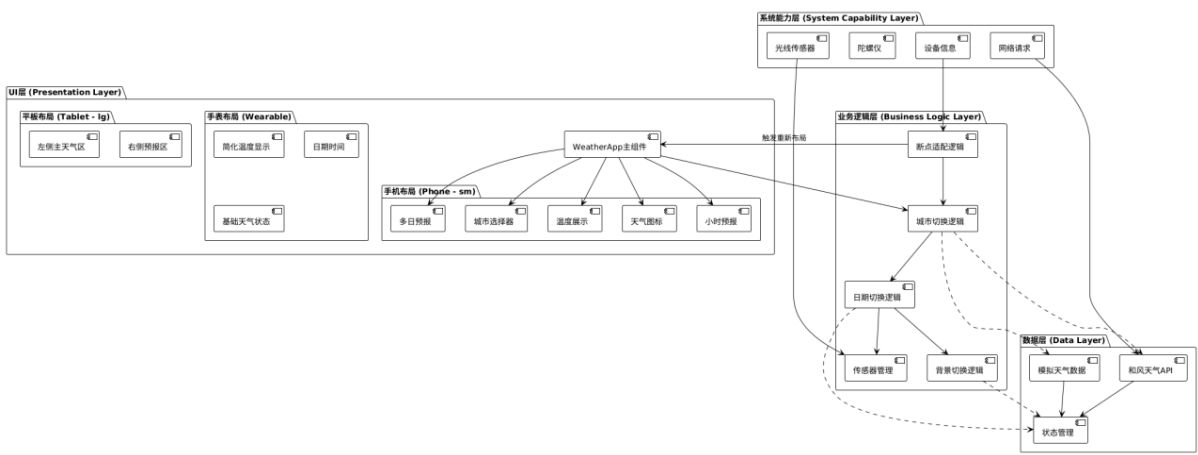


图 2-2 架构图

应用从入口启动后，首先进入核心的 `Index` 页面，它是整个应用的调度中心，向下驱动四大核心模块协同工作。设备类型判断模块会自动识别运行设备，并加载对应的布局方案——手表端使用轻量化布局，手机 / 平板端则采用更丰富的显示布局，实现了跨设备的适配能力；天气数据模块支持双数据源接入，既可以调用和风天气 API 获取真实数据，也能使用模拟数据进行开发调试，保证了开发与线上环境的兼容性。

传感器模块和分布式数据模块为应用提供了扩展能力与数据持久化支持。传感器模块接入了光线传感器与陀螺仪传感器，可根据环境光线自动切换界面主题、结合陀螺仪数据实现交互效果，提升用户体验；分布式数据模块基于 `KVStore` 存储实现数据的本地持久化，支持用户设置、城市列表等数据的跨会话保存，让应用在重启后也能恢复用户状态，保障了使用的连续性。

7.3 顺序图

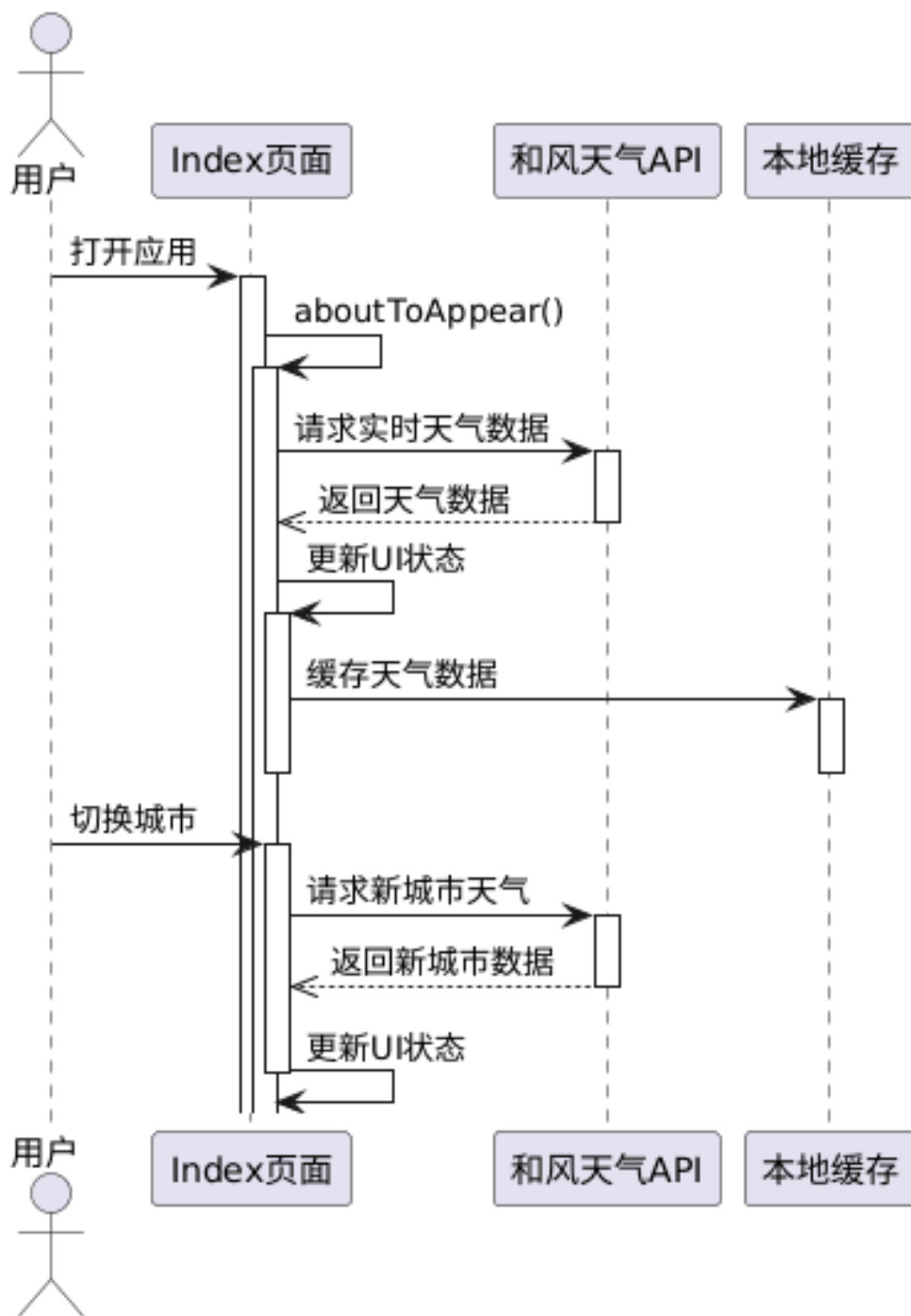


图 2-3 顺序图

这张时序图清晰展示了天气应用从用户打开应用到首次加载数据的完整交互流程。当用户打开应用后，**Index** 页面触发生命周期方法 `aboutToAppear()`，随后主动向和风天气 **API** 发起实时天气数据请求；在成功获取数据后，页面会更新 **UI** 状态并将数据写入本地缓存，实现了数据的展示与持久化。

时序图同时呈现了用户切换城市时的交互流程。当用户触发切换城市操作后，**Index** 页面会向和风天气 **API** 请求新城市的天气数据，获取数据后再次更新 **UI** 状

态，为用户展示目标城市的天气信息。整个流程以 **Index** 页面为核心，串联起用户操作、API 请求与数据缓存，保证了数据获取与展示的连贯性。

技术分析报告

多端适配分析

- 手机布局特点：垂直排列，信息完整，支持横向滚动
- 手表布局特点：简洁紧凑，核心信息突出
- 设备判断：通过设备类型 API 自动识别

日期切换机制分析

- 状态管理：使用@State 装饰器管理选中日期索引
- 数据联动：选中日期后同步更新温度、天气、背景、小时预报
- 用户体验：选中高亮、平滑过渡

传感器集成分析

- 光线传感器应用场景：环境光感知，可用于自动调节界面亮度
- 数据采集频率：实时响应，精度满足应用需求

背景图片切换分析

- 判断优先级：雷 > 晴 > 雨 > 阴 > 云
- 视觉效果：根据天气类型展示对应氛围背景

问题与解决

问题描述	解决方案	解决结果
日期点击无反应	将 Column 改为 Button 组件，绑定 onClick 事件	已解决
背景图片不切换	调整天气判断顺序，雷字优先判断	已解决

问题描述	解决方案	解决结果
横向滚动失效	添加 Scroll 组件包裹，设置 scrollable(0)	已解决
城市选择器溢出	添加 Scroll 组件支持横向滚动	已解决

实验总结

实验收获

- 掌握了 HarmonyOS ArkTS 声明式 UI 开发
- 理解了多端适配的设计思路和实现方法
- 学会了传感器数据采集和实时更新
- 掌握了状态管理和事件处理机制
- 理解了鸿蒙分布式能力的基础概念

技术要点

- @State 状态管理实现 UI 响应式更新
- ForEach 循环渲染列表数据
- Scroll 组件实现横向滚动
- Button 组件绑定点击事件
- 条件渲染实现多端适配
- 传感器 API 集成

考核指标

考核项	评分标准	得分
功能完整性	天气应用功能完整，城市/日期切换正常	25 分
代码质量	代码结构清晰，类型定义完整	20 分
多端适配	手机/手表界面自适应良好	15 分
传感器集成	光线传感器数据采集正常	15 分

考核项	评分标准	得分
用户体验	交互流畅，视觉效果良好	15 分
实验报告	报告结构完整，内容详实	10 分

教师评价

项目	评价
实验完成情况	
技术掌握程度	
创新点	
综合评价	
成绩	

教师签名： _____

日期： _____